

Justificación de Diseño DDD — SecureBankito

1. Enfoque general de diseño

La solución se diseñó como una Prueba de Concepto de un sistema bancario seguro dividido en tres dominios principales: IAM, BankingCore y VIP Assets. Cada dominio se implementa como un microservicio independiente, con responsabilidad funcional clara y una base de datos propia, respetando el principio de autonomía entre dominios.

Desde la perspectiva de Domain-Driven Design táctico, la solución separa la lógica de negocio en entidades, objetos de valor, repositorios, casos de uso y servicios de dominio. La seguridad no se modela únicamente como una capa externa de autorización, sino como parte de las reglas del dominio. Por eso, las decisiones de acceso se basan en etiquetas de seguridad como `clearance_level`, `integrity_level` y `role`, emitidas por IAM dentro del token JWT.

En la implementación actual, la equivalencia entre los niveles del enunciado y el código es la siguiente:

- BRONZE representa el Bronce.
- SILVER representa Plata.
- PLATINUM representa Oro.

Esta equivalencia permite mantener los nombres usados en el código y, al mismo tiempo, explicar la solución usando el lenguaje del enunciado del proyecto.

2. Dominio IAM

El dominio IAM tiene la responsabilidad de gestionar usuarios, credenciales y etiquetas de seguridad. Su agregado principal es Usuario, ya que concentra la identidad del sujeto y sus atributos de acceso.

Usuario se modela como Entity y Aggregate Root porque tiene identidad propia, representada por `UserId`, y puede existir a lo largo del tiempo aunque cambien algunos de sus atributos, como el nombre de usuario, la contraseña o el nivel de seguridad. Dos usuarios no se comparan por todos sus atributos, sino por su identidad.

Los objetos `UserId`, `Username`, `Password`, `PasswordHash`, `SecurityLevel`, `ClearanceLevel`, `IntegrityLevel` y `UserRole` se modelan como Value Objects porque no poseen ciclo de vida independiente dentro del dominio. Su propósito es encapsular reglas de validación y representar conceptos del lenguaje ubicuo. Por ejemplo, `Username` valida que el nombre no esté vacío y cumpla restricciones de formato; `Password` válida reglas mínimas de complejidad; `SecurityLevel` agrupa el nivel de confidencialidad y el nivel de integridad del usuario.

`SecurityLevel` se considera un Value Object porque representa una combinación de atributos de seguridad. No tiene identidad propia; dos niveles de seguridad son equivalentes si tienen el mismo `clearance` y el mismo `integrity`. Esta decisión permite que las reglas de seguridad sean tratadas como parte del modelo de dominio y no como simples cadenas de texto dispersas en el sistema.

`AuthService` se modela como Domain Service porque realiza una operación del dominio que no pertenece de forma natural a una sola entidad: generar un token JWT con las etiquetas de seguridad del usuario. Este token contiene información como identificador del usuario, `clearance`, nivel de integridad y rol, lo que permite que otros microservicios apliquen reglas de seguridad sin compartir directamente la base de datos de IAM.

3. Dominio BankingCore

El dominio `BankingCore` gestiona cuentas bancarias y transacciones. Este dominio es el responsable de aplicar la regla de integridad Biba antes de ejecutar movimientos financieros.

`CuentaBancaria` se modela como Aggregate Root porque representa el estado financiero principal del dominio: el saldo de una cuenta asociada a un propietario. La cuenta tiene identidad propia mediante `AccountId` y mantiene su saldo mediante el objeto de valor `Money`.

Transacción se modela como Entity porque posee identidad propia mediante `TransactionId` y representa un evento financiero auditable. Aunque dos transacciones tengan el mismo monto y las mismas cuentas involucradas, siguen siendo transacciones distintas porque tienen identificadores y momentos de creación diferentes. Además, la transacción conserva su estado (`COMPLETED` o `REJECTED`) y una posible razón de rechazo, lo que permite mantener rastro de auditoría.

Money se modela como Value Object porque representa una cantidad monetaria inmutable y validada. Sus operaciones add y subtract no modifican directamente la instancia original, sino que devuelven nuevos valores. Además, impide montos negativos, lo cual protege una regla básica del dominio financiero.

AccountId, OwnerId, TransactionId, CreatedAt, TransactionStatus y RejectionReason también se modelan como Value Objects porque representan conceptos simples pero importantes del dominio. Encapsulan validaciones como formato UUID, estado permitido de una transacción o existencia de una razón de rechazo válida.

“ProcesadorTransferencias” se modela como Domain Service porque la transferencia entre cuentas involucra varias entidades y reglas que no pertenecen exclusivamente a una sola cuenta ni a una sola transacción. Este servicio coordina la búsqueda de cuentas, la validación de seguridad, la actualización de saldos y el registro de la transacción.

La validación Biba ocurre antes de modificar saldos. En la implementación actual, el servicio compara el integrity level del usuario/proceso solicitante contra el integrity level del dueño de la cuenta destino, obtenido desde IAM. Si el solicitante tiene menor integridad que el receptor, la transacción se registra como REJECTED y se retorna un error 403. Esto permite demostrar el escenario No Write Up solicitado en la PoC.

4. Dominio VIP Assets

El dominio VIP Assets gestiona activos financieros de alto valor. Estos activos representan información confidencial de nivel Oro, por lo que su lectura está protegida mediante Bell-LaPadula.

“ActivoInversión” se modela como Aggregate Root y Entity porque representa un activo financiero con identidad propia. Cada activo tiene un identificador único (AssetId), un nombre (AssetName) y un valor económico (AssetValue). Aunque dos activos tengan el mismo nombre o el mismo valor, siguen siendo activos distintos por su identidad.

AssetId, AssetName y AssetValue son Value Objects. AssetId encapsula la validez del identificador; AssetName valida que el nombre del activo no esté vacío y tenga longitud mínima; AssetValue usa un valor decimal y evita valores negativos. Estas decisiones reducen la posibilidad de que datos inválidos entren al dominio.

La regla Bell-LaPadula se aplica al intentar leer activos VIP. En la implementación actual, el middleware “requirePlatinum” valida que el token JWT contenga clearance_level =

PLATINUM, equivalente al nivel Oro del enunciado. Si un usuario Bronce o Plata intenta listar o consultar activos VIP, el sistema retorna 403 Forbidden. Esto demuestra la regla No Read Up, ya que un sujeto de menor nivel de confidencialidad no puede leer información clasificada en un nivel superior.

5. Separación entre dominios y seguridad

La solución mantiene los tres dominios separados para evitar acoplamiento innecesario. IAM gestiona la identidad y emite tokens; BankingCore consume las etiquetas de integridad para proteger operaciones financieras; y VIP Assets consume las etiquetas de confidencialidad para proteger la lectura de activos Oro.

Esta separación favorece el principio de Zero Trust, porque cada microservicio valida el token recibido y aplica sus propias reglas de autorización. El API Gateway funciona como punto de entrada, pero la seguridad relevante también se valida dentro de cada servicio. De esta manera, aunque una petición llegue a un microservicio, todavía debe pasar por las reglas propias del dominio.

6. Relación con los modelos de seguridad

La regla Biba se implementa en BankingCore como una protección de integridad. Su objetivo es evitar que un proceso o usuario de menor integridad pueda modificar información asociada a un nivel superior. En la PoC, esto se demuestra cuando un usuario Bronce intenta depositar dinero en una cuenta asociada a un usuario Oro y el sistema rechaza la operación con 403 Forbidden.

La regla Bell-LaPadula se implementa en VIP Assets como una protección de confidencialidad. Su objetivo es evitar que usuarios de menor clearance puedan leer información clasificada en niveles superiores. En la PoC, esto se demuestra cuando un usuario Plata intenta listar activos VIP de nivel Oro y el sistema responde con 403 Forbidden.

7. Comentario sobre decisiones actuales

La implementación actual cumple el propósito de la PoC porque permite demostrar errores 403 controlados para Biba y Bell-LaPadula. Sin embargo, existe una decisión de diseño importante: actualmente la integridad usada para validar Biba se obtiene del dueño de la cuenta destino en IAM, no de un atributo propio de la cuenta bancaria. Esta decisión

simplifica la PoC y evita modificar el modelo de base de datos, pero puede evolucionar en una versión posterior para que CuentaBancaria tenga su propio `required_integrity_level`.

En una implementación más alineada con la especificación, el nivel de integridad requerido debería pertenecer directamente a la cuenta bancaria. En ese caso, CuentaBancaria tendría un Value Object adicional, por ejemplo “RequiredIntegrityLevel”, y el “ProcesadorTransferencias” compararía el nivel de integridad del solicitante contra el nivel requerido por la cuenta destino.